



Tecnicatura Universitaria en Programación a Distancia

Programación III

Trabajo Práctico Integrador

“Food Store” - Sistema de Gestión de Pedidos de comida - Aplicación web full stack

Alumno:

Santiago Octavio Varela (santiago.varela@tupad.utn.edu.ar) - Comisión 13

Fecha de Entrega: 29 de Junio de 2026

Índice

1. [Marco Teórico](#)
2. [Decisiones Técnicas y Arquitectura](#)
3. [Dificultades y Soluciones](#)
4. [Bibliografía](#)
5. [Anexo: Links a repositorio GitHub y Video](#)

Marco Teórico

Tecnologías del Backend

Java (Versión 25)

Lenguaje de programación robusto, orientado a objetos y fuertemente tipado que sirve como pilar del backend. Su adopción garantiza portabilidad a través de la Máquina Virtual de Java (JVM). En el proyecto se aplican características de Java moderno, entre ellas las expresiones Lambda y la API de Streams para el procesamiento declarativo de colecciones, el uso de objetos contenedores Optional para evitar excepciones de tipo NullPointerException y, por último, la implementación de Records para la definición concisa de objetos inmutables de transferencia de datos.

Spring Boot

Spring Boot es un framework de código abierto que simplifica la configuración y el despliegue de aplicaciones basadas en Spring empresarial. Su arquitectura se fundamenta en dos principios esenciales:

- **Inversión de Control (IoC) y el ApplicationContext:** Actúa como el contenedor central encargado de gestionar automáticamente el ciclo de vida y la conexión de las dependencias de la aplicación (Beans).
- **Auto-configuración y Starters:** Automatiza la inclusión y configuración de bibliotecas comunes mediante paquetes preestablecidos (ej. spring-boot-starter-web), reduciendo el código repetitivo y eliminando las configuraciones manuales en XML.

Jakarta Persistence API (JPA) y Spring Data JPA

Jakarta Persistence API (JPA) es la especificación estándar de Java para el Mapeo Objeto-Relacional (ORM), que reduce la brecha entre el paradigma orientado a objetos de la aplicación y la estructura relacional de la base de datos. Mediante el uso de Spring Data JPA y la abstracción JpaRepository, el sistema interactúa de manera directa con el motor de persistencia, accediendo a operaciones CRUD automatizadas y consultas derivadas (métodos Query) sin necesidad de escribir SQL nativo explícito.

Proyecto Lombok

Biblioteca de utilidades orientada a optimizar la productividad y legibilidad del código Java en tiempo de compilación. A través de anotaciones semánticas (@Data, @Getter, @Setter, @NoArgsConstructor, @AllArgsConstructor, etc.), genera automáticamente el código repetitivo (métodos de acceso, constructores y métodos estructurales toString, equals y hashCode), manteniendo limpias las clases de dominio.

OpenAPI y Swagger

OpenAPI y Swagger son el estándar utilizado para estructurar contratos y documentar las interfaces de programación de aplicaciones de forma viva e interactiva. Con la integración de springdoc-openapi-starter, el backend expone automáticamente una interfaz gráfica web (Swagger UI) que detalla de forma interactiva las rutas (endpoints), parámetros esperados, códigos de respuesta HTTP y esquemas de datos (DTOs).

Tecnologías del Frontend

HTML5 (HyperText Markup Language)

HTML es el lenguaje de marcado estándar utilizado como herramienta básica para la construcción de las interfaces web de la aplicación. Su función principal radica en definir la organización semántica y la anatomía jerárquica del contenido renderizado por el navegador (el "esqueleto" del sitio). En el proyecto se implementan las especificaciones de HTML5 Semántico a través de etiquetas estructuradas con propósito propio (como <header>, <nav>, <main>, <section>, <article> y <footer>), lo cual optimiza la legibilidad del código para el equipo de desarrollo y eleva los estándares de accesibilidad para tecnologías asistivas, favoreciendo también el posicionamiento orgánico en motores de búsqueda (SEO). Asimismo, se explota la arquitectura de formularios nativos e inputs de tipo específico para garantizar una validación inicial y una recopilación estructurada de datos en el cliente.

CSS3 (Cascading Style Sheets)

Lenguaje de diseño encargado de modelar la capa estelar de presentación visual, estilos y maquetación de la interfaz de usuario. Mediante la adopción de selectores estructurados y propiedades visuales modernas, se desacopla completamente el diseño gráfico de la estructura del documento. La arquitectura estético-estructural del frontend se sustenta en tres pilares avanzados:

- **Modelos de Layout Unidimensional y Bidimensional (Flexbox y CSS Grid):** Utilizados para la distribución inteligente, alineación predecible y ordenamiento dinámico del espacio entre componentes de software (como la grilla del catálogo de productos).
- **Estrategia de Diseño Adaptativo (Responsive Web Design / Media Queries):** Mecanismo que emplea reglas de consulta basadas en características del dispositivo (ancho, orientación y escala) bajo la filosofía de desarrollo Mobile First, garantizando una experiencia de usuario consistente tanto en terminales móviles como en escritorios de alta resolución.
- **Metodología BEM (Block, Element, Modifier):** Convención de nomenclatura profesional de clases CSS aplicada para mitigar conflictos de especificidad en proyectos a gran escala, asegurando un ecosistema de estilos modular, escalable y altamente mantenible.

Vite

Herramienta de construcción y entorno de desarrollo rápido orientada a aplicaciones web modernas. Su motor aprovecha la disponibilidad de los módulos ES nativos (ESM) en el navegador para agilizar notablemente el ciclo de recarga (Hot Module Replacement - HMR) y delega el empaquetado final a herramientas optimizadas, logrando una velocidad superior frente a herramientas de compilación tradicionales.

TypeScript

TypeScript es un superconjunto (o superset) tipado de JavaScript que introduce tipado estático opcional, interfaces de programación y herramientas avanzadas de análisis de código en tiempo de desarrollo. Debido a que los navegadores web no tienen soporte nativo para ejecutar archivos .ts, la herramienta implementa un proceso previo de transpilación (mediante el compilador tsc), el cual valida los contratos de tipos establecidos y degrada el código a JavaScript estándar compatible con el motor del cliente. Dentro de la arquitectura de la aplicación, TypeScript actúa como un elemento de control de calidad dividido en los siguientes pilares de desarrollo:

- **Tipado Estático y Seguridad en Tiempo de Compilación:** A diferencia de JavaScript (su tipado es dinámico y expone fallas lógicas directamente al usuario final en tiempo de ejecución), TypeScript restringe y audita la asignación de variables, parámetros y retornos de funciones. El motor del lenguaje obliga a definir estructuras estrictas, eliminando anomalías de cohesión de datos y errores silenciosos antes de empaquetar el producto.
- **Contratos de Software mediante Interfaces y Type Aliases:** Empleado de forma sistemática para reflejar fielmente en el cliente las estructuras de datos devueltas por la API REST de Spring Boot. Mediante la declaración de estructuras formales (interface), el frontend cuenta con un modelo predictivo estricto de las entidades de negocio (como productos, categorías y pedidos). Esto habilita el autocompletado inteligente en el entorno de desarrollo (IDE) y reduce las fallas por inconsistencia o tipeo erróneo en los nombres de las propiedades.
- **Mecanismos de Control e Inferencia (Type Assertions y Type Guards):** En escenarios complejos como la captura de datos de formularios web mediante FormData, el sistema implementa declaraciones explícitas de tipo para guiar de forma segura las expectativas del compilador. Complementariamente, se aplican estrategias de estrechamiento de tipos (Type Narrowing) utilizando validadores lógicos como typeof e instanceof. Esto garantiza que los flujos de código ejecuten métodos específicos únicamente cuando se ha corroborado la identidad real del dato.
- **Interacción Segura con el Modelo de Objetos del Documento (DOM):** El lenguaje transforma la incertidumbre del manejo del DOM en contratos tipados y estricta robustez. Al seleccionar componentes de la interfaz de usuario por medio de selectores generales (document.querySelector), TypeScript asigna tipos genéricos o específicos a las etiquetas HTML (HTMLFormElement, HTMLButtonElement, KeyboardEvent, etc.), otorgando visibilidad exclusiva sobre las propiedades del elemento y también impidiendo accesos inválidos. Adicionalmente, el compilador obliga al equipo a resolver de forma explícita la nulidad potencial de los

componentes mediante estructuras de control condicional o el operador de encadenamiento opcional (?.), protegiendo la aplicación frente a fallas del hilo principal.

- **Modelado del Flujo Asíncrono y Estado de la API (Fetch API):** Al romper el aislamiento de datos locales para consumir los servicios del backend por medio de llamadas en red, TypeScript neutraliza el tipado genérico implícito (any) de las promesas de la Fetch API. Tras resolver las llamadas con la sintaxis de control asíncrono async/await, el cuerpo de la respuesta del servidor es explícitamente casteado y asignado a su tipo de interfaz correspondiente. Esto asegura que los datos recuperados dinámicamente cumplan estrictamente con las reglas del sistema y gestionen las mutaciones de la interfaz en los estados de carga, éxito y manejo de errores gráficos.

Arquitectura y Conceptos Clave Aplicados

Arquitectura de Diseño por Capas

Para garantizar el desacoplamiento, la mantenibilidad y la separación clara de responsabilidades, el backend se estructura siguiendo un patrón arquitectónico modular dividido en tres niveles principales:

- **Capa de Controlador (API REST / Controllers):** Puntos de entrada expuestos al protocolo de red que se encargan exclusivamente de interceptar peticiones HTTP, mapear los datos entrantes y despachar la respuesta apropiada al cliente.
- **Capa de Servicio (Business Logic / Services):** Componente aislado donde reside la lógica de negocio nuclear del negocio, coordinando las reglas operativas, validaciones específicas y transformaciones de datos.
- **Capa de Repositorio (Data Access / Repositories):** Interfaz encargada del acceso y la persistencia de datos relacionales, comunicándose directamente con la base de datos a través del ORM.

Patrón Data Transfer Object (DTO)

Patrón de diseño de software concebido con la finalidad de transferir información entre procesos de forma segura y eficiente. El uso de DTOs separa las Entidades persistentes de la base de datos de los datos expuestos públicamente por la API REST. Esto mitiga vulnerabilidades comunes de sobreexposición (Over-posting), optimiza el ancho de banda de red enviando únicamente los campos requeridos por el frontend y flexibiliza la evolución interna del esquema de datos.

Manejo Global de Excepciones (@ControllerAdvice)

Componente arquitectónico centralizado orientado a interceptar y gestionar los errores imprevistos producidos en cualquier capa de la aplicación. Mediante controladores de asesoramiento (@ExceptionHandler), el sistema intercepta las excepciones de negocio o validación de datos para transformarlas en respuestas HTTP con formatos JSON

estandarizados y códigos de estado semánticos apropiados (ej. 400 Bad Request, 404 Not Found), garantizando una experiencia de integración predecible y limpia para el frontend.

Consumo de APIs Asíncronas

Mecanismo implementado en el frontend nativo del navegador para interactuar con servicios externos mediante llamadas en red sin bloquear la ejecución del hilo principal de la interfaz de usuario. A través de la sintaxis moderna `async/await`, el sistema gestiona los flujos asíncronos y asegura un manejo de errores robusto. Además, el desarrollo contempla de forma estricta el ciclo de vida del estado visual de la interfaz de usuario, sustituyendo dinámicamente entre estados de Carga (Loading) mediante deshabilitación de componentes, Éxito (Success) al renderizar la respuesta del servidor, y Fallo (Error) para la retroalimentación del usuario final.

Decisiones Técnicas y Arquitectura

Para el desarrollo del sistema Food Store, se adoptó una arquitectura cliente-servidor claramente desacoplada, priorizando la mantenibilidad, el rendimiento y la seguridad del código. A continuación, se detallan las decisiones técnicas más relevantes:

Arquitectura del Backend (Spring Boot)

El servidor fue estructurado bajo una arquitectura en capas (controllers, services, repositories, etc.) como la venimos trabajando durante la cursada, asegurando el principio de responsabilidad única.

- **Uso intensivo del Patrón DTO (Data Transfer Object):** En lugar de exponer directamente las entidades JPA/Hibernate, se implementaron records de Java 17+. Se crearon DTOs específicos de entrada (CreateDto , EditDto) para validar rígidamente los payloads que llegan al servidor (usando Jakarta Validation), y DTOs de salida (ResponseDto) para aplanar la información. Esto resolvió problemas de referencias cíclicas y previno la sobreexposición de datos sensibles (como la contraseña en la entidad Usuario).
- **Manejo Global de Excepciones:** Se implementó una clase anotada con @RestControllerAdvice para centralizar la captura de errores. Gracias a esto, si un recurso no existe o una validación de formulario falla, el servidor no arroja un stacktrace en crudo, sino que devuelve un JSON estandarizado con el código HTTP correspondiente (ej. 404 Not Found o 400 Bad Request), facilitando su manejo por parte del Frontend.
- **Seguridad y Cifrado Ligerito:** Para cumplir con la historia de usuario de encriptación de contraseñas sin incurrir en la sobrecarga de configurar y acoplar toda la arquitectura de Spring Security, se optó por un enfoque pragmático. Se diseñó un método aislado utilizando el algoritmo nativo SHA-256 (MessageDigest), logrando un hashing unidireccional seguro directamente en la capa de servicios.
- **Carga Inicial de Datos (Data Seeding):** Para facilitar el despliegue y testeo sin depender de scripts .sql externos o cargas manuales vía Postman, se desarrolló la clase DataInitializer para cargar los datos adaptador del “consumible” ofrecido en las consignas de este trabajo práctico integrador (pese a que en la versión para Spring Boot no estaba). Utilizando la anotación @PostConstruct e inyección de dependencias por constructor, esta clase puebla automáticamente la base de datos en memoria (H2) con usuarios maestros, categorías, productos y pedidos de prueba durante el arranque de la aplicación.
- **Diseño de API RESTful y Verbos HTTP:** Se diseñó una API rigurosamente apegada al estándar REST, utilizando sustantivos para definir los recursos (por ejemplo /api/productos, /api/categorias, etc.) y apoyándose en la semántica de los métodos HTTP (GET, POST, PUT, DELETE) para las operaciones CRUD, evitando ensuciar las URLs con acciones.
- **Implementación de Actualizaciones Parciales (PATCH):** Un punto arquitectónico destacable fue la diferenciación entre reemplazos completos y actualizaciones parciales. Para la gestión de pedidos, en lugar de forzar un PUT masivo que podría sobrescribir accidentalmente los detalles de la compra, se desarrolló el endpoint

específico `PATCH /api/pedidos/{id}/status` . Esto permitió realizar una mutación de estado ligera, segura y semánticamente correcta.

- **Validación de Entradas (Input Validation):** Todos los endpoints transaccionales (creación y edición) fueron securizados a nivel de controlador mediante la anotación `@Valid` . En conjunto con los DTOs, esto delegó las validaciones de negocio (campos obligatorios, nulos, etc.) al framework de Spring (Jakarta Validation), rechazando peticiones malformadas antes de que alcancen la capa de servicios.
- **Endpoint Personalizado de Autenticación:** Se construyó una ruta ad-hoc `POST /api/usuarios/login` separada de la creación de recursos estándar, diseñada puramente para procesar las credenciales en texto plano, contrastarlas con el hash SHA-256 de la base de datos y devolver un DTO de respuesta saneado que alimenta la sesión del frontend.

Arquitectura del Frontend (TypeScript + Vite)

El frontend fue desarrollado sin librerías externas para cumplir con el desafío de dominar los fundamentos web, apoyándose en TypeScript y el empaquetador Vite.

- **Tipado Estricto (Interfaces compartidas):** Se crearon interfaces en TypeScript (ej. `IUser` , `PedidoDto`) que son un espejo exacto de los DTOs del backend. Esto mitigó el desfase de contratos en el consumo de la API nativa `fetch()` , permitiendo que el IDE alerte en tiempo de compilación sobre propiedades inexistentes o `undefined`.
- **Proxy Inverso y Evasión de CORS:** Al trabajar en un entorno desacoplado (Vite en el puerto 5173 y Spring Boot en el 8080), surgían bloqueos por políticas de Cross-Origin Resource Sharing (CORS). Se solucionó interceptando el tráfico desde la configuración (`vite.config.ts`), estableciendo un proxy que redirige internamente cualquier petición `/api` hacia el backend. Esto también permitió escribir llamadas `fetch` mucho más limpias y agnósticas del entorno de producción.
- **Seguridad de Rutas (Guards Locales):** La autenticación y persistencia de sesión se manejó mediante el `LocalStorage` del navegador. Se desarrolló una función utilitaria (`checkAuthUser`) que actúa como un guardián de ruta (Route Guard), interceptando el ciclo de vida de la página para validar la existencia de la sesión y el rol del usuario, ejecutando redirecciones inmediatas si un "Usuario" normal intenta acceder a zonas restringidas del "Admin".
- **Manejo Dinámico de Recursos Estáticos:** Se centralizó una lógica para la carga de imágenes que soporta tanto URLs externas (`http...`) como assets locales (`/images/`). Adicionalmente, se saneó el DOM mediante la inyección nativa de `EventListeners` en JavaScript, los cuales asignan automáticamente una imagen de reemplazo (`placeholder.webp`) en caso de que ocurra una falla de red, garantizando que la UI nunca se rompa.
- **Diseño Modular con BEM y Flexbox:** En general, el modelo de caja fue estandarizado utilizando únicamente Flexbox. La estilización se basó en la metodología BEM (Bloque-Elemento-Modificador) y en variables de entorno CSS (`:root`), permitiendo una interfaz 100% responsiva, semántica y escalable (facilitando la implementación de modos oscuros y cambios de branding).

Interfaces Principales del Sistema

A continuación, se presentan las capturas de las vistas más representativas donde confluyen las decisiones arquitectónicas mencionadas:

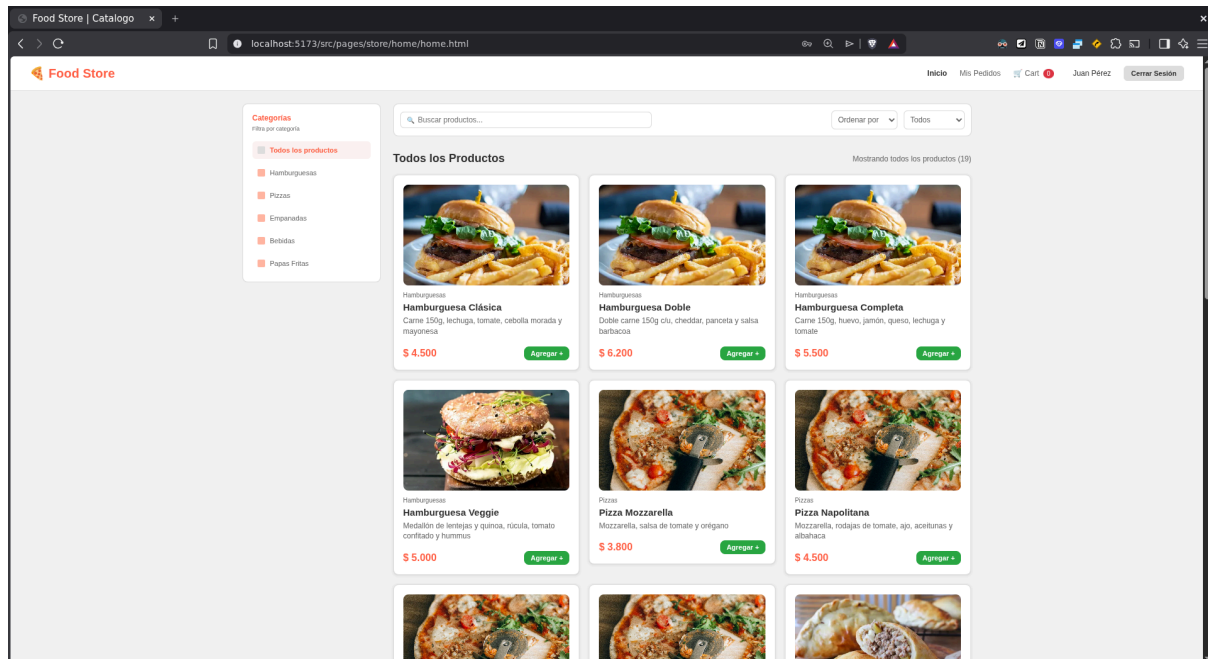
Captura 1: Formulario de Registro o Login demostrando el diseño de componentes:

The image displays two screenshots of a web application interface for 'Food Store'. Both screenshots show a white form centered on an orange background.

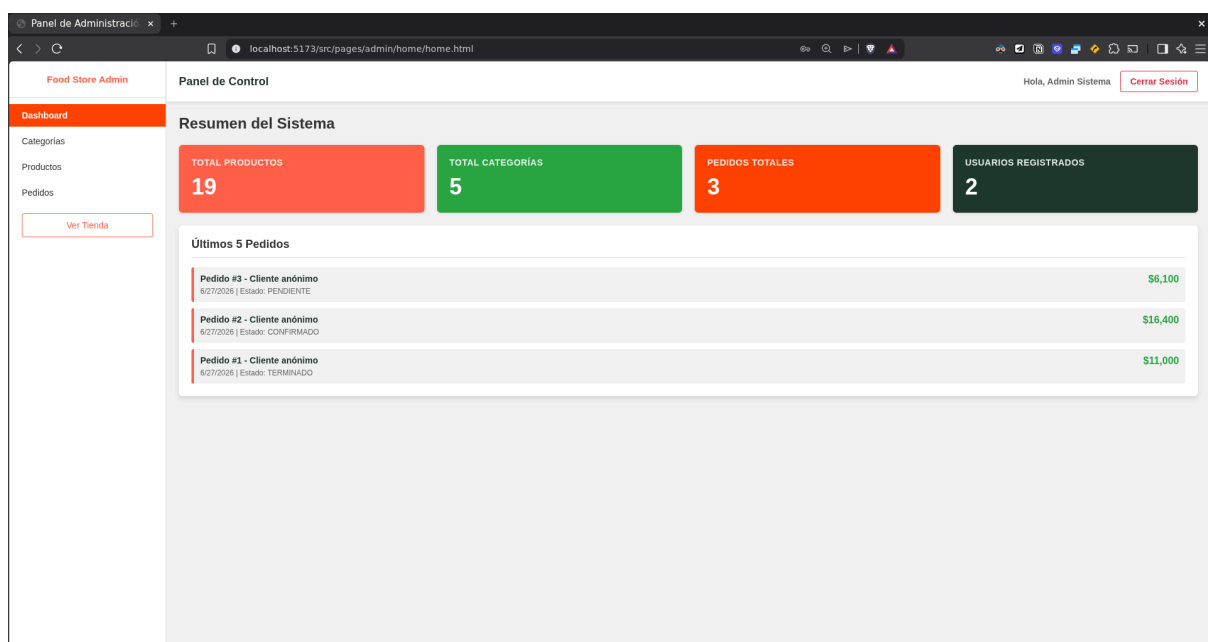
Top Screenshot: Login Form
The browser tab is 'Login Food Store' and the URL is 'localhost:5173/src/pages/auth/login/login.html'. The form is titled 'Food Store Iniciar Sesión'. It contains two input fields: 'Email' (with placeholder 'ejemplo@correo.com') and 'Contraseña' (with placeholder 'Ingresa tu contraseña'). Below these is an orange 'Ingresar' button. A link '¿No tienes cuenta? [Regístrate](#)' is positioned below the button. At the bottom, there is a small text block: 'Cuentas de prueba: Admin: admin@food.com / 123456, Cliente: cliente@food.com / cliente123'.

Bottom Screenshot: Register Form
The browser tab is 'Registro | Food Store' and the URL is 'localhost:5173/src/pages/auth/registro/registro.html'. The form is titled 'Food Store Registro'. It contains five input fields: 'Nombre' (placeholder 'Ej: Juan'), 'Apellido' (placeholder 'Ej: Pérez'), 'Email' (placeholder 'ejemplo@correo.com'), 'Celular (Opcional)' (placeholder 'Ej: 1123456789'), and 'Contraseña' (placeholder 'Mínimo 6 caracteres'). Below these is an orange 'Registrarse' button. A link '¿Ya tienes cuenta? [Inicia sesión](#)' is positioned below the button.

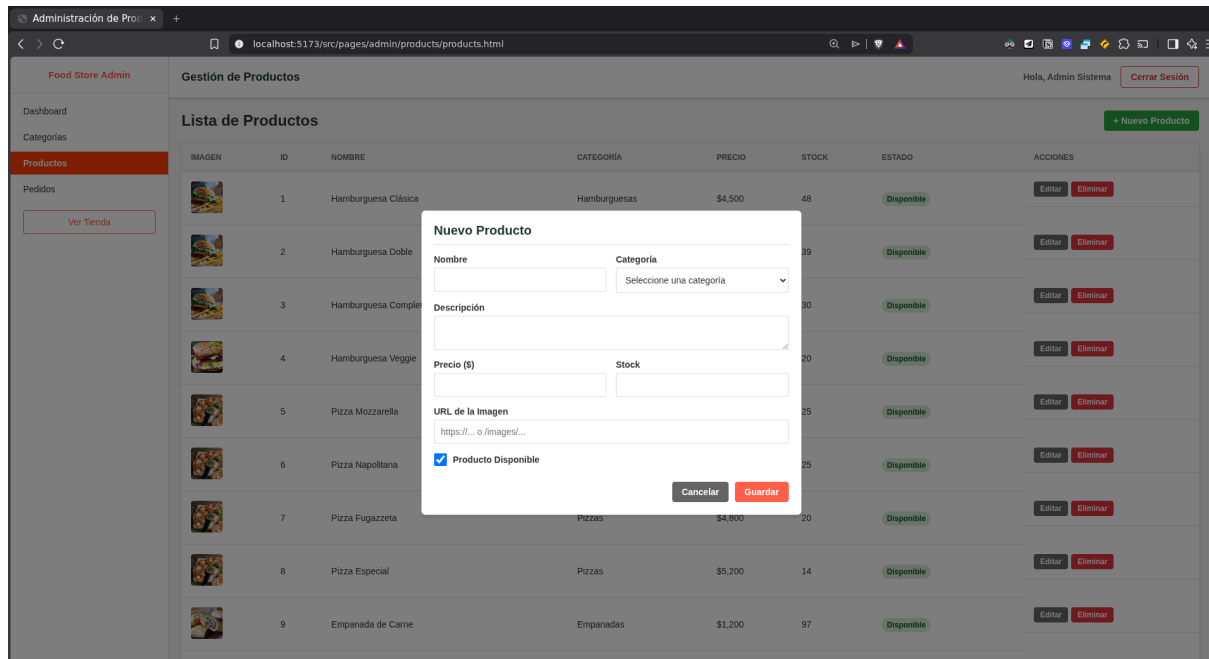
Captura 2: Vista del Catálogo/Tienda del Cliente mostrando los productos



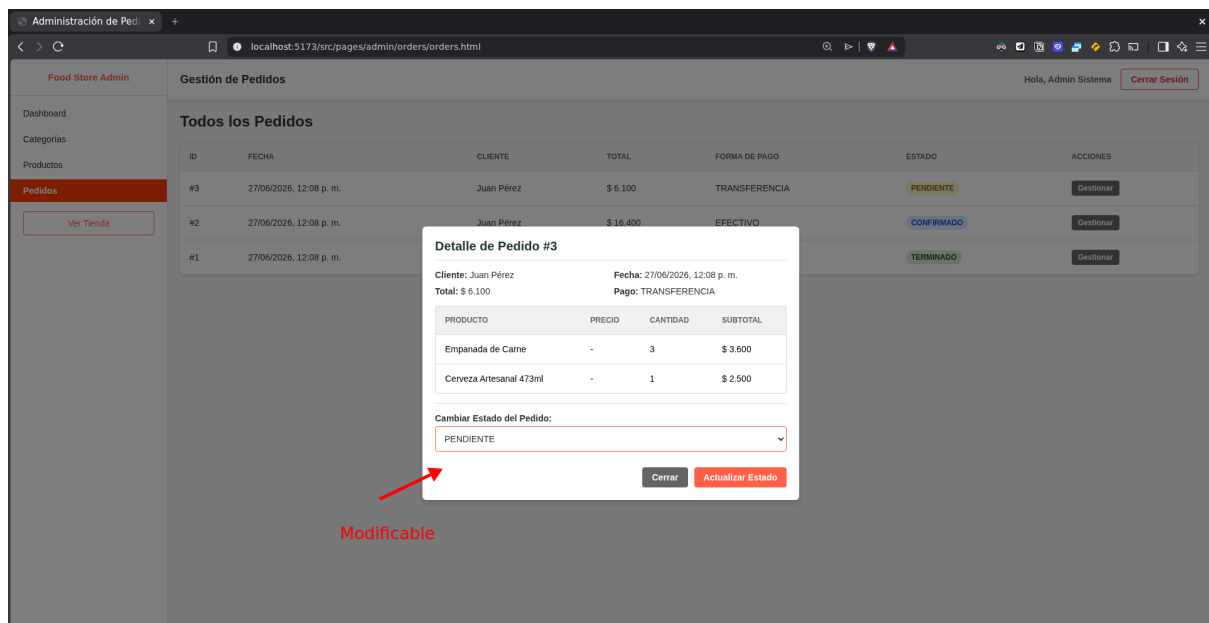
Captura 3: Dashboard Principal del Panel de Administración con las estadísticas en tiempo real



Captura 4: Modal nativo de CRUD en la Gestión de Productos mostrando sus diferentes propiedades (Nuevo Producto es similar al modal de “Editar” en cualquiera de los productos preexistentes)



Captura 5: Modal nativo de CRUD en la Gestión de Pedidos mostrando sus diferentes estados



Dificultades y Soluciones

Respecto a las dificultades y soluciones, para empezar caben algunos comentarios generales. Un rasgo importante fue el hecho de que gran parte de las historias de usuario correspondientes a los diferentes Sprints ya estaban validadas, porque la estructura está constituida por los avances escalonados que hicimos en los diferentes trabajos prácticos y parciales. En ese sentido, no se hicieron los sprints a rajatabla en su orden cronológico desde el comienzo del desarrollo de este trabajo práctico integrador, sino que se aislaron las historias de usuario faltantes para completar el trabajo.

Por lo tanto, una primera dificultad fue la sistematización y reclasificación del estado de todas las historias de usuario, siguiendo cuáles estaban completas y cuáles no dentro de sus sprints correspondientes, para luego implementar en el código las funcionalidades faltantes. Finalmente, se ejecutaron pruebas finales integrales para revalidar tanto las funcionalidades previas como las nuevas, para completar este trabajo integrador.

Una segunda y tercera dificultad general, aunque menos relevante, fue la de la curva de aprendizaje que conlleva el uso simultáneo de múltiples tecnologías y su ejecución concurrente de los entornos de desarrollo (Vite para frontend y Gradle para backend). La única solución plausible a esto fue volver a los materiales y videos de estudio, a la vez que buscaba nueva información en documentación oficial para resolver problemas puntuales.

A continuación detallaré algunas dificultades y soluciones puntuales de las que fui tomando nota para completar esta sección del trabajo:

Frontend

Gestión del Entorno y UI/UX

Dificultad: Retomar el frontend tras semanas de inactividad y encontrar un volumen imprevisto de maquetación, estilos y lógica pendiente desde el primer parcial.

Solución: Implementación de un sistema de priorización para aislar la lógica funcional de los detalles estéticos, asegurando el desarrollo de los flujos críticos antes de completar la maquetación secundaria.

Dificultad: Desconocimiento sobre la ejecución simultánea de los entornos de desarrollo (Vite para frontend y Gradle para backend).

Solución: Configuración del paquete concurrently mediante npm para unificar los scripts de arranque, permitiendo levantar y visualizar los logs de ambos servicios de forma concurrente desde una única terminal.

Integración y Consumo de API

Dificultad: Primera experiencia conectando un frontend con un backend propio y ejecutando peticiones HTTP mediante la API fetch.

Solución: Adopción de fetch mediante funciones asíncronas (async/await). Para asegurar la robustez, se estandarizó el uso de bloques try/catch en cada petición para capturar

excepciones de red, y se mitigó la incongruencia de datos tipeando estrictamente las respuestas JSON del servidor usando interfaces de TypeScript (DTOs) compartidas en el Frontend.

Dificultad: Incertidumbre sobre el estado y la configuración de los endpoints disponibles en el backend.

Solución: Utilización de clientes HTTP (Postman o Insomnia) para testear, documentar y validar las respuestas y códigos de estado de cada ruta del backend de forma aislada antes de integrarlas en el código cliente.

Modelado de Datos

Dificultad: Desfasaje en la estructura de datos por desarrollo asíncrono. El backend exigía 5 campos para la entidad Usuario, mientras que el template de registro del frontend recolectaba y enviaba solo 3 (nombre, email, contraseña).

Solución: Sincronización de los contratos de datos (DTOs). Se ampliaron los formularios HTML/CSS del frontend para incluir los inputs faltantes y se ajustó el mapeo en la función de registro para asegurar que el payload del POST coincidiera con las restricciones de validación de Spring Boot.

Backend

Integridad de Datos y Validación

Dificultad: Cumplir con la Definition of Done (DoD) respecto a "validaciones específicas", un requerimiento no abordado en fases previas (TP10).

Solución: Integración de Spring Boot Bean Validation (org.springframework.boot:spring-boot-starter-validation). Aplicación de anotaciones (@NotNull, @NotBlank) en los DTOs y @Valid en los controladores. Esto garantiza el rechazo de datos inválidos antes de la capa de servicio, protegiendo la base de datos contra fallos de validación del lado del cliente.

Seguridad y Encriptación

Dificultad: Ambigüedad de alcance. La HU-026 exigía la encriptación de contraseñas, lo cual generaba fricción con la directiva general de que el proyecto "no requería seguridad real".

Solución: Implementación de hashing unidireccional utilizando el algoritmo SHA-256 nativo de Java (MessageDigest). Esto permitió cumplir con la historia de usuario encriptando los datos sensibles en la base de datos de manera aislada y segura, evitando por completo la enorme sobrecarga de instalar y configurar la dependencia completa de Spring Security.

Comunicación de Red

Dificultad: Bloqueo de peticiones HTTP por parte del navegador debido a políticas de seguridad al comunicar el puerto del cliente con el del servidor.

Solución: Configuración global de las políticas CORS (Cross-Origin Resource Sharing) mediante la creación de la clase de configuración `config/WebConfig.java`. Se habilitaron explícitamente los orígenes, métodos (GET, POST, etc.) y headers necesarios para el frontend.

Consistencia de API

Dificultad: Desfasaje en las estructuras de datos. Al desarrollar el backend de forma aislada, los objetos JSON devueltos por la API presentaban inconsistencias con la información esperada y renderizada por las interfaces del frontend.

Solución: Refactorización de la capa de presentación del backend mediante el patrón DTO de salida (Response DTOs). Se ajustaron los controladores para formatear y devolver únicamente los campos requeridos por los componentes del frontend, eliminando la sobreexposición de atributos de las entidades originales.

Bibliografía

- Universidad Tecnológica Nacional. (2026). Programación 3: *Material teórico general*. Tecnicatura Universitaria en Programación a Distancia. Recuperado del campus virtual institucional.
- Microsoft. (s.f.). *MessageDigest Class*. Microsoft Learn. Recuperado el 27 de junio de 2026, de <https://learn.microsoft.com/en-us/dotnet/api/java.security.messagedigest?view=net-android-35.0>
- Mozilla. (s.f.). *Cross-Origin Resource Sharing (CORS)*. MDN Web Docs. Recuperado el 27 de junio de 2026, de <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>
- npm. (s.f.). *concurrently*. Recuperado el 27 de junio de 2026, de <https://www.npmjs.com/package/concurrently>
- Oracle. (s.f.). *MessageDigest (Java Platform SE 8)*. Recuperado el 27 de junio de 2026, de <https://docs.oracle.com/javase/8/docs/api/java/security/MessageDigest.html>
- Spring. (s.f.). *Spring Boot Reference Documentation*. Recuperado el 27 de junio de 2026, de <https://docs.spring.io/spring-boot/index.html>

Anexo: Links a repositorio GitHub y Video

- Link al repositorio en GitHub: https://github.com/santiagovOK/Varela_Santiago_final-prog3
- Link para ver el video explicativo: <https://youtu.be/iPpf2Dc6y9Y>
- Link B para ver el video explicativo:
<https://drive.google.com/file/d/1qVi9tkZsW7vVKzdnn4HUKPuNAdQb3p0k/view?usp=sharing>